
High Level Design Specification (HLDS)

for

Asynchronous FIFO

Version 6.0

Prepared by Asritha Manju Immadisetty

Bhavani Gunda

Vijay kotagiri

ECE-593: Fundamentals of Pre-Silicon Validation – Venkatesh Patil

03/07/2023

Table of Contents

Table of Contents	ii
Revision History	iii
1. Introduction.....	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions.....	1
1.4 Product Scope.....	2
1.5 References	3
2. Overall Description.....	3
2.1 Product Perspective	3
2.2 Product Functions.....	4
2.3 User Classes and Characteristics	5
2.4 Tools and Software.....	6
2.5 Design and Implementation Constraints.....	6
2.6 Assumptions and Dependencies	7
3. External Interface Requirements	8
3.1 Hardware Interfaces.....	8
3.2 Software Interfaces	9
4. Product Features	9
4.1 Write and Read pointer	9
4.2 FIFO Memory.....	10
4.3 Two Flop Synchronizer	11
4.4 Timing Diagram	12
5. Logic Design	13
5.1 <Your work Directory Structure>	13
5.2 <Design modules>.....	13
5.3 <SystemVerilog abstraction Features used>	13
5.4 <Simulation, Tools, Directory Structure>	13
6. Verification	14
6.1 Testbench Components.....	16
6.2 Testbench Style	16
6.3 Testing Strategies	18
6.4 Test case scenarios	19
6.5 Self checking with scoreboard.....	19
7. Simulation Results.....	20
7.1 SV Simulation result.....	20
7.2 UVM Simulation result.....	21
Summary.....	22
Appendix A: Glossary.....	22
Appendix B: FIFO Depth Calculation.....	24

Revision History

Name	Date	Reason For Changes	Version
Asritha	02/03/24	Mile stone 1[until section4]	1.0
Bhavani	02/11/24	Mile stone 2[added sectoion5]	2.0
Vijay	02/17/24	Mile stone 3[added section 6.1,6.2,6.3,6.4-SV]	3.0
Asritha	02/25/24	Mile stone 4[added 6.1,6.2,6.3,6.4,6.5-UVM]	4.0
Bhavani	03/03/24	Mile stone 5[edited section6 and summary]	5.0
Vijay	03/07/24	Final Project submission [final check]	6.0

1. Introduction

1.1 Purpose

The purpose of this High-Level Design Specifications (HLDS) document is to clearly provide the detailed description about Asynchronous FIFO (First-In-First-Out) for maintainability and documentation of the design. As a high-level reference guide for the interactions between the modules, this document also aims to assist in identifying conflicts before coding. It provides requirements for understanding the project, documentation of architecture, design rationale, guidance for development, scalability, extensibility.

The scope of this document is to describe features of FIFO, architectural overview, major components and modules, data flow and control flow, external interface requirements, design patterns for implementation and verification plan to test the design.

1.2 Document Conventions

Times New Roman – Size 18- Bold is used to mention headings

Times New Roman – Size 14- Bold is used to mention sections

Times New Roman – Size 12- Bold is used to mention sub-sections

Abbreviations

FIFO : First In- First Out

ASIC : Application-Specific Integrated Circuit

FPGA : Field Programmable Gate Arrays

EDA : Electronic Design Automation

SoC : System-on-Chip

HDL : Hardware Description Languages

1.3 Intended Audience and Reading Suggestions

Professionals, academics, and practitioners working in the fields of FIFO implementation, asynchronous systems, and digital design are among the readers of these works. These publications include insightful information and useful methodologies that engineers, scholars, and anybody interested in digital circuit design, particularly those working with asynchronous designs, may benefit from this.

1.4 Product Scope

The asynchronous FIFO design is meticulously crafted to address the challenges associated with asynchronous systems, with a specific focus on pointer comparisons within the FIFO structure. The design incorporates simulation and synthesis techniques to achieve optimal functionality and performance

Benefits of the Design

1. Improved Performance: The intent of the asynchronous FIFO design is to implement asynchronous pointer comparisons in order to improve overall system performance. As a result, data transfer latency is decreased, resulting in quicker and more effective communication.
2. Increased Reliability: Timing fluctuations may cause problems for asynchronous systems. By addressing these issues, the design makes the FIFO structure more durable across a range of situations and more reliable while operating asynchronously.

Objectives of the Design

1. Efficient Data Transfer: Facilitating effective data transmission in asynchronous systems is one of the main objectives. Through the use of sophisticated synthesis and simulation methods, the design seeks to maximize the FIFO structure for dependable and smooth data flow.
2. Reduced Latency: Minimizing latency is a key objective. Because asynchronous pointer comparisons help to shorten data transmission delays, the architecture is appropriate for situations where low latency is essential.

Goals of the Design

1. Optimization for Various Applications: The design strives to be flexible enough to accommodate a wide range of digital system applications. It seeks to offer a FIFO structure that is flexible enough to accommodate a range of use cases and can be optimized for varied circumstances.
2. Data Integrity and Synchronization: Achieving synchronization and data integrity are essential objectives. In order to synchronize the asynchronous FIFO operation with the overall system needs, the design places a strong emphasis on preserving the integrity of sent data.

Integration into Bigger Designs

1. System-on-Chip (SoC): The asynchronous FIFO is essential for controlling data flow among various SoC components. The SoC's total performance and functionality are enhanced by its effective functioning.
2. Central Processing Unit (CPU): An essential component of a CPU for managing data flow between pipeline stages is the asynchronous FIFO. Improving this part helps increase the overall speed and effectiveness of the CPU.

3. **Network Chips:** To manage data packets asynchronously, network chips must have an asynchronous FIFO architecture. In network communication, it helps to lower latency and improve data transmission reliability.

1.5 References

1. <https://vlsiverify.com/verilog/verilog-codes/asynchronous-fifo/>
2. <https://vlsiuniverse.blogspot.com/2013/09/synchronization-schemes.html>
3. <https://www.verilogpro.com/asynchronous-fifo-design/>
4. Clifford E. Cummings, Sunburst Design, “Simulation and Synthesis Techniques for Asynchronous FIFO Design” http://www.sunburst-design.com/papers/CummingsSNUG2002SJ_FIFO1.pdf
5. Clifford E. Cummings, Peter Alfke, “Simulation and Synthesis Techniques for Asynchronous FIFO Design with Asynchronous Pointer Comparisons” http://www.sunburstdesign.com/papers/CummingsSNUG2002SJ_FIFO2.pdf
6. Karthik Rao, Nitin Goel, Prashant Bhargava, “Design Transition from Sync to Async: Design and Verification Challenges”, <https://www.design-reuse.com/articles/31630/design-transition-from-sync-to-async-design-and-verification-challenges.html>
7. <https://verificationguide.com/uvm/uvm-testbench-architecture/>
8. Venkatesh Patil-“ECE-593-Lecture Slides”.

2. Overall Description

2.1 Product Perspective

Origin

The concept of asynchronous FIFO has its roots in the broader field of digital system design and communication. Asynchronous FIFOs were developed to address the challenges associated with handling data in systems where components operate independently and may not share a common clock signal. The origin can be traced to the need for efficient and reliable data transfer in asynchronous systems, where timing variations and lack of synchronization between components pose significant hurdles.

The asynchronous FIFO design, likely evolved in response to the growing demand for solutions that could ensure orderly data flow in digital systems characterized by asynchronous operation. The design principles and techniques presented in the document likely emerged from a combination of theoretical advancements and practical considerations in the realm of asynchronous digital circuit design.

Context

The context of asynchronous FIFO is characterized by the need to address timing variations and lack of synchronization between subsystems within a digital system. The scope of this asynchronous FIFO includes data synchronization, buffering to store data temporarily, flow control mechanism to manage the rate of data transfer, avoiding metastability issues by providing a

structured way to transfer data asynchronously, addressing latency differences by buffering the data, clock domain crossing without introducing timing violations. The asynchronous FIFO is essential for controlling data flow as a component in larger systems like SoCs, CPUs and Network chips.

Block Diagram

The simple block diagram of asynchronous FIFO is shown in fig.1. would include components such as Write Pointer, Read Pointer, fifo memory, Two 2-flop synchronizers.

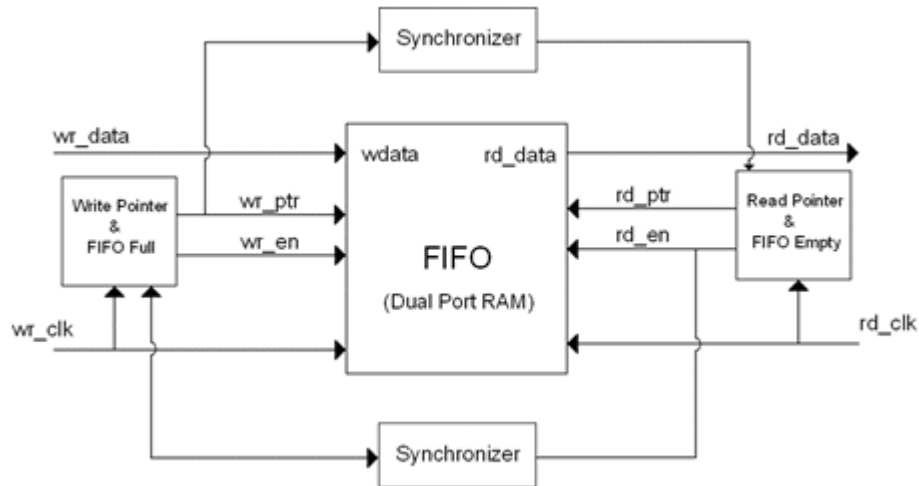


Figure 1: Simple Block diagram of asynchronous FIFO

2.2 Product Functions

Asynchronous FIFO is a type of digital circuit or data structure used in electronic design to manage the flow of data between two asynchronous processes or clock domains.

Data Buffering: Data buffering between two subsystems with distinct clock zones is the principal function of an asynchronous FIFO. When the input and output systems have different rates of data generation and consumption, it permits the temporary storage of data. In the design read clock frequency is 250Mhz and write clock frequency is 100Mhz.

Dual-Port Memory: Asynchronous FIFO often makes use of dual-port memory, in which the output system reads data from the other port while the input system writes data to the first port. This makes it possible to read and write simultaneously. Fifo memory module contains these operations.

Pointer system: To maintain track of the read and write locations inside the FIFO, the architecture uses a pointer system. Techniques for managing these pointers in an asynchronous environment where clock domains function independently are referred to as asynchronous pointer comparisons. wptr and rptr are the write pointer and read pointer respectively.

Conditions of Empty and Full: The FIFO is equipped with sensors to determine whether it is empty or full. These prerequisites are essential for maintaining flow control and avoiding overflow or data loss. The write operation is usually halted until there is available space in the FIFO when it is full, (signal full=1), and the read operation may also be halted until there is data (signal empty=1)..

Handshaking Protocols/Synchronization: Asynchronous FIFOs need handshaking protocols to coordinate data transfers between the input and output systems. This guarantees error-free and dependable data flow even in the absence of a world clock signal. This is done by using 2 flop synchronizers.

Simulation Techniques: The behavior of the Asynchronous FIFO under various situations is modeled and analyzed using simulation techniques. This aids in discovering any problems and making the design more suitable for performance and reliability.

2.3 User Classes and Characteristics

- **Digital System Designers:** Asynchronous FIFOs enable reliable data buffering between components with different clock rates, utilizing dual-port memory and pointer mechanisms for efficient data transfer in complex digital systems.
- **Communication System Developers:** Valuable in communication systems with varying data rates, Asynchronous FIFOs manage empty/full conditions and employ handshaking protocols, ensuring seamless data flow and minimizing the risk of data loss or overflow.
- **Embedded System Engineers:** Asynchronous FIFOs provide a solution for interfacing diverse components in embedded systems with independent clocks, facilitating smooth data transfer without requiring a global clock signal.
- **ASIC and FPGA Designers:** Crucial for ASIC and FPGA designers, Asynchronous FIFOs involve a synthesis process translating high-level design into hardware description language (HDL), facilitating efficient implementation on specific hardware platforms.
- **Real-Time System Developers:** In real-time systems, Asynchronous FIFOs support precise and reliable data transfer through simulation techniques, enabling developers to model and optimize system behavior under various conditions for performance and reliability.
- **IoT Device Manufacturers:** As a communication bridge in IoT devices, Asynchronous FIFOs accommodate the asynchronous nature of data streams from diverse sensors and actuators, enhancing the integration and efficiency of IoT systems.
- **High-Performance Computing Applications:** Asynchronous FIFOs facilitate efficient data exchange in high-performance computing scenarios where processing units operate at distinct speeds. Dual-port memory and handshaking protocols contribute to maintaining synchronized and reliable data flow.

2.4 Tools and Software

- **Environment Description:**

The software operates in an environment tailored for the simulation and synthesis of Asynchronous FIFO designs, with a focus on managing pointers in an asynchronous context.

- **Hardware Platform:**

The software is intended for hardware description languages (HDL) and FPGA/ASIC design environments. It is compatible with various hardware platforms commonly used in digital circuit design, including FPGA development boards and ASIC testing environments.

- **Operating System and Versions**

The software is typically platform-independent, as it is primarily used within FPGA and ASIC design tools that support multiple operating systems. Compatible operating systems include Windows, Linux, and macOS, ensuring flexibility for design engineers using different development environments.

- **EDA (Electronic Design Automation) Tools**

The software seamlessly integrates with EDA tools commonly used in digital circuit design. Mentor Questa is used to simulate the model.

- **Tools for Synthesis**

- During the synthesis stage, the software works in unison with synthesis instruments to convert the Asynchronous FIFO's high-level design into a hardware description that can be implemented on certain platforms. Accurate translation of the design into hardware is ensured by compatibility with synthesis tools such as Xilinx Vivado Synthesis, cadence, mentor Questa sim, Synopsys Design Compiler. Mentor questa is used to synthesis this design in particular.

- **Version Control Frameworks**

The program interfaces with version control programs (like Git) to support teamwork in development and preserve a history of modifications made to the design. Asynchronous FIFO design changes may be tracked and managed by users, guaranteeing version consistency and effective teamwork.

2.5 Design and Implementation Constraints

- The minimum depth of asynchronous FIFO is 130 bytes. It has been given adjusted to the next nearest powers of 2. So now, the depth of our FIFO is 256 (2 to the power 8).
- The data burst is given as 150 according to the specification.
- The parameter width is considered as 8bit wide.
- The width of asynchronous FIFO is assumed to be 8 bits.
- All the above-mentioned values can be parameterized for varying size.
- The FIFO will be using active low reset.
- The data will be written at positive edge of wclk and data will be read at the positive edge of rclk.

- When the fifo contains 256 elements full becomes high and when fifo contains no elements empty signal becomes high indicating the full and empty conditions for the fifo.

2.6 Assumptions and Dependencies

Assumptions

- The design assumes the presence of multiple clock domains, and the need to transfer data asynchronously between these domains is a fundamental requirement.
- The assumption that dual-port memory is available or can be efficiently implemented. Dual-port memory is often a key component in asynchronous FIFO designs.
- The design assumes proper synchronization of control signals and handshaking protocols to avoid race conditions and ensure reliable data transfer.
- The assumption that the pointer mechanism, used for tracking read and write positions in the FIFO, is stable and reliable under various conditions.

Dependencies

- Design may depend on the use of specific HDLs like Verilog or VHDL for describing the hardware components and behavior.
- Dependencies on simulation tools like Model Sim, VCS, or others to simulate the behavior of the asynchronous FIFO design and validate its functionality.
- Dependencies on proper clock constraints and considerations to ensure accurate and reliable operation across different clock domains.
- The design may depend on certain characteristics of the input data, such as data width, data rate, and patterns, which may influence the design choices.
- Dependencies on specific requirements for the output, including data rate, latency, and other performance metrics.

3. External Interface Requirements

3.1 Hardware Interfaces

Block Diagram:

The detailed block diagram of asynchronous FIFO is shown in fig.2.

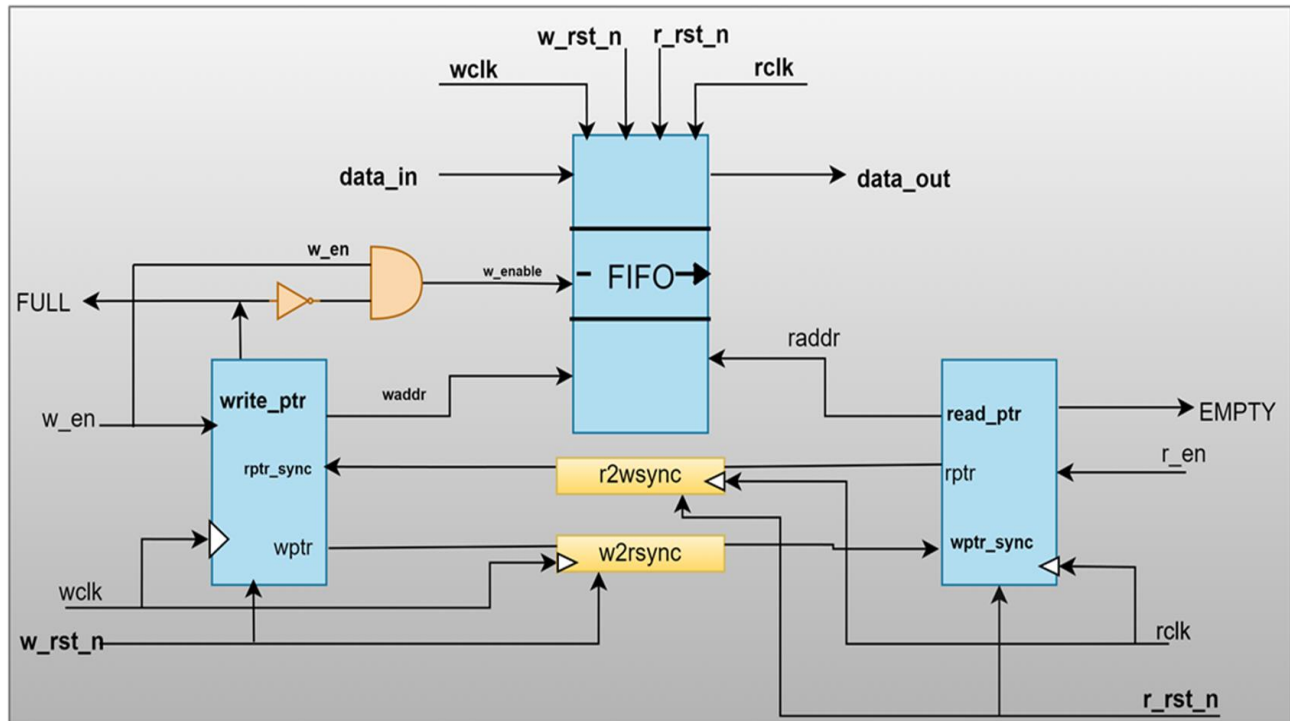


Figure 2: Block diagram of asynchronous FIFO

Components

The asynchronous FIFO block diagram would include components such as Write Pointer & full module, Read Pointer & empty module, FIFO memory and two synchronizers.

RTL Modules

The RTL description would consist of modules corresponding to each functional block in the block diagram.

top.sv– top level wrapper module

fifo_mem.sv – the FIFO memory buffer that is accessed by the write and read clock domains

r2wsync.sv– 2 flip-flop synchronizer to synchronize read pointer to write clock domain

w2rsync.sv – 2 flip-flop synchronizer to synchronize write pointer to read clock domain

rprr– synchronous logic in the read clock domain to generate FIFO empty condition

wptr – synchronous logic in the write clock domain to generate FIFO full condition

Connections - Interconnections Between Modules:

The modules would be interconnected as per the functional requirements of the asynchronous FIFO design. For instance, the Write Pointer Module will be connected to the memory Module, read pointer module will be connected to the memory module. The Status and Control Logic will be connected to both Write and Read Pointer Modules.

3.2 Software Interfaces

Simulation Software

The asynchronous FIFO concept may be simulated using Mentor QuestaSim.

Synthesis Tools

For implementation on FPGA or ASIC platforms, the RTL (Register-Transfer Level) description of the asynchronous FIFO may be translated into a gate-level netlist using synthesis tools like Synopsys Design Compiler or Xilinx Vivado. But in this HLDS design is implemented in Questa sim.

OperatingSystem

The synthesis and simulation tools' needs or the designers' preferences may influence the operating system selection. Widely used operating systems include Windows and Linux.

Programming Language

System Verilog is used for describing the asynchronous FIFO design at the RTL level.

4. Product Features

4.1 Write pointer and Read pointer

In a synchronous FIFO design, one way to determine whether a FIFO is full or empty is to use separate count register to track the number of entries in the FIFO. This requires the ability to both increment and decrement the counter, potentially on the same clock. The same technique cannot be used in an asynchronous FIFO, however, because two different clocks will be needed to control the counter.

Instead, the asynchronous FIFO design uses a different technique (also derived from synchronous FIFO design) of using an additional bit in the FIFO pointers to detect full and empty. In this scheme, full and empty is determined by comparing the read and write pointers. The write pointer always points to the next location to be written; the read pointer always points to the current FIFO entry to be read. On reset, both pointers are reset to zero. The FIFO is empty when the two pointers (including the extra bit) are equal. It is full when the MSB of the pointers are different, but remaining bits are equal. This FIFO pointer convention has the added benefit of low access latency.

As soon as data has been written, the FIFO drives read data to the FIFO data output port, hence the receive side does not need to use two clocks (first set a read enable, then read the data) to read out the data. Warning can be generated after FIFO is full.

4.2 FIFO memory

A FIFO (First-In-First-Out) memory is a type of data storage that follows the First-In-First-Out principle, meaning that the first data element to enter the memory is the first one to be removed. It operates as a buffer or queue, managing data in a sequential manner. FIFO memory finds extensive use in various digital systems and applications where data needs to be stored and retrieved in a structured and ordered fashion.

FIFO memory stores data in a sequential order, resembling a queue. The first data element to be stored is the first to be retrieved. It typically employs read and write pointers to manage the data flow. The write pointer indicates the location where new data is written, and the read pointer indicates the location from which data is read. ensure data integrity by preserving the order in which data is received. This is crucial in applications where the temporal sequence of data elements matters. In systems with asynchronous components, FIFO memory can serve as a synchronization point. It allows asynchronous subsystems to communicate without being directly dependent on each other's timing.

4.3 Two flop synchronizer

FIFO needs a 2 two-flop synchronizers, one for write pointer and one for read pointer.

Modern VLSI designs with intricate architectures and multiple clock sources often involve interactions between different clock domains. In the absence of proper synchronization, the risk of metastability events arises, potentially leading to chip failure. Synchronizers play a crucial role in mitigating the effects of metastability, especially when signals traverse clock domain boundaries. Understanding metastability and designing effective synchronizers are essential skills in preventing catastrophic failures.

Synchronizers act as a key mechanism for tolerating metastability by converting asynchronous signals or signals from different clock domains into the recipient clock domain. While it's impossible to completely avoid metastability, synchronizers significantly reduce the chances of its occurrence. An ideal synchronizer is reliable, ensuring a high Mean Time Between Failures (MTBF), exhibits low latency from source to destination domain, and has minimal area and power impact. When a signal transitions between clock domains, the synchronizer absorbs any potential metastability effects, preventing downstream logic from entering a metastable state induced by the first flip-flop in the new clock domain.

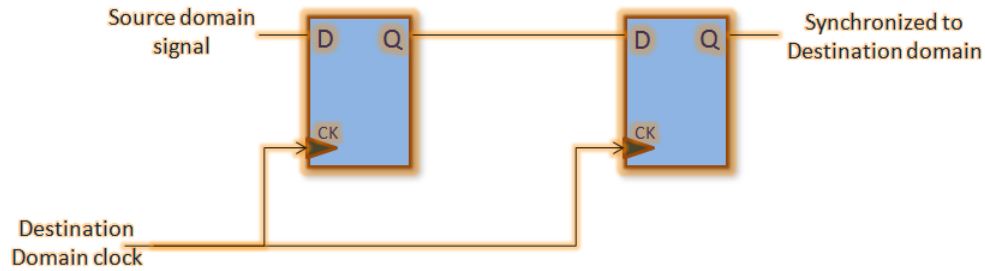


Figure 4: Two flop synchronizer

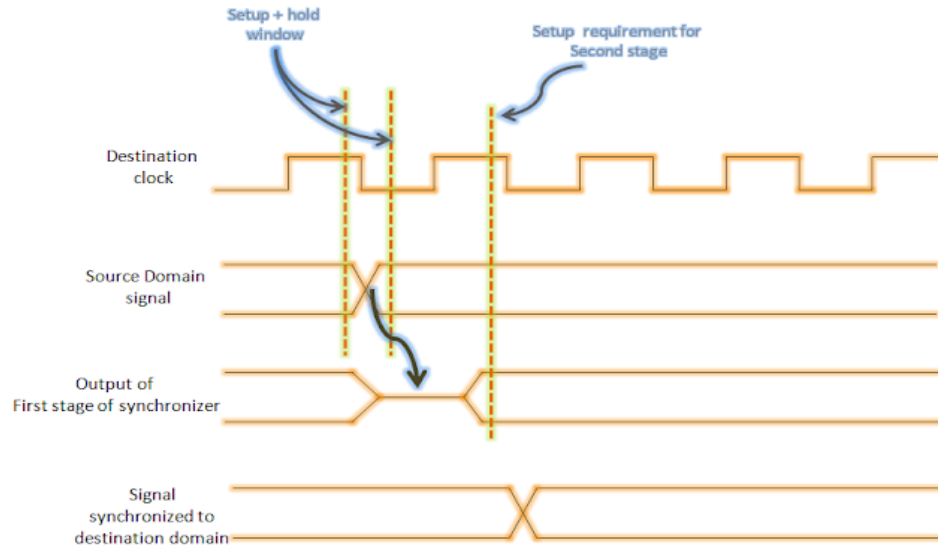


Figure 5: Timing diagram for two flop synchronizer

This synchronization scheme involves a chain of two or more flip-flops in the destination clock domain, providing a simple and common solution. In the case of a two-flip-flop synchronizer, the output (Q2) stabilizes 1 or 2 cycles later than the input, resolving metastability within one clock period. Although the first stage (Q1) may occasionally go metastable, as long as resolution occurs before the next clock edge, the second stage's output (Q2) maintains valid logic levels. The theoretical maximum latency for synchronizing an asynchronous signal is 2 clock cycles.

4.4 Timing Diagram

Read operation

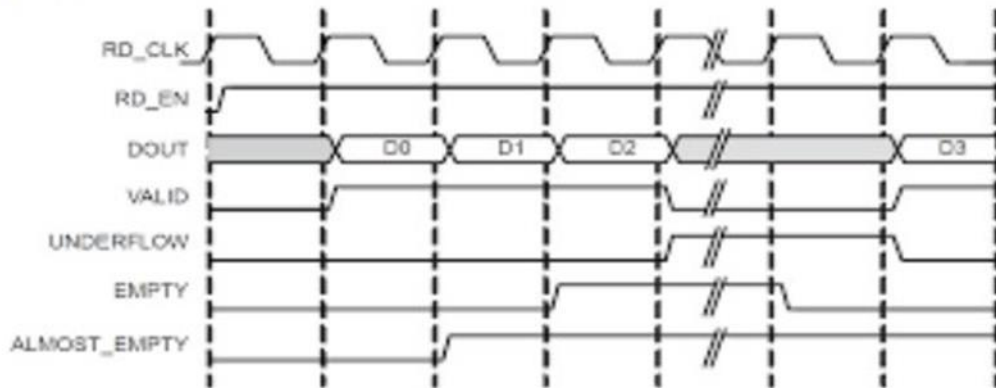


Figure 5: Timing diagram for read operation of FIFO

Write operation

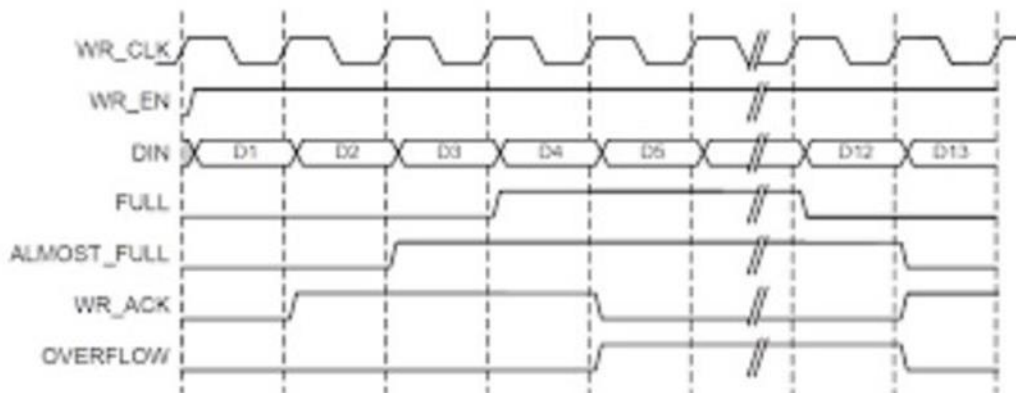


Figure 6: Timing diagram for write operation of FIFO

5. Logic Design

5.1 Github work Directory

https://github.com/gunda-1325/pre-si_async_fifo_t3

5.2 Design modules

<what will be your hardware coding style, how many modules, how many, how they will be connected to each other, how will they interact with each other and to the outer world>

The design contains 5 modules namely write pointer handler & full module(w_ptr), read pointer handler & empty module(r_ptr), fifo memory module(fifo_mem), read to write synchronizer(r2wsync), write to read synchronizer(w2rsync).

- w_ptr module: It monitors the write pointer and generates full condition with respect to write clock(wclk) and write reset(w_rst).
- r_ptr module: It monitors the read pointer and generates empty condition with respect to read clock(rclk) and read reset(r_rst).
- fifo_mem: allows to write using w_en and read using r_en with respect to wclk and rclk simultaneously with two different clock frequencies.
- r2wsync: synchronizes the rptr values by delaying for 2 cycles and sends it as an input to the w_ptr module
- w2rsync: synchronizes the wptr values by delaying for 2 cycles and sends it as an input to the r_ptr module

All these 5 modules are instantiated in the top module and given appropriate port connections. The top module is instantiated in the testbench to drive stimulus and produce outputs with the help of interface.

5.3 SystemVerilog abstraction Features used

- Parameterization
- Interfaces
- Classes

5.4 Simulation, Tools, Directory Structure

Mentor-QuestaSim is used to simulate the design. The wave form contains all the input and output signals on the top module such as wclk, rclk, r_rst, w_rst, w_en, t_rn, data_in, data_out, full, empty. All the burst details will be printed on the transcript based on the read enable or write enable.

6. System Verilog Verification

The verification of this design is done using both system Verilog and UVM. The detailed explanation of these testbench architectures is explained clearly in further sections of this document.

- In system Verilog verification, class-based verification is done by implementing the system Verilog verification environment from scratch. The verification environment contains different transaction class, generator, driver, monitor and scoreboard and coverage. The block diagram of the system Verilog environment is shown in fig. 7.

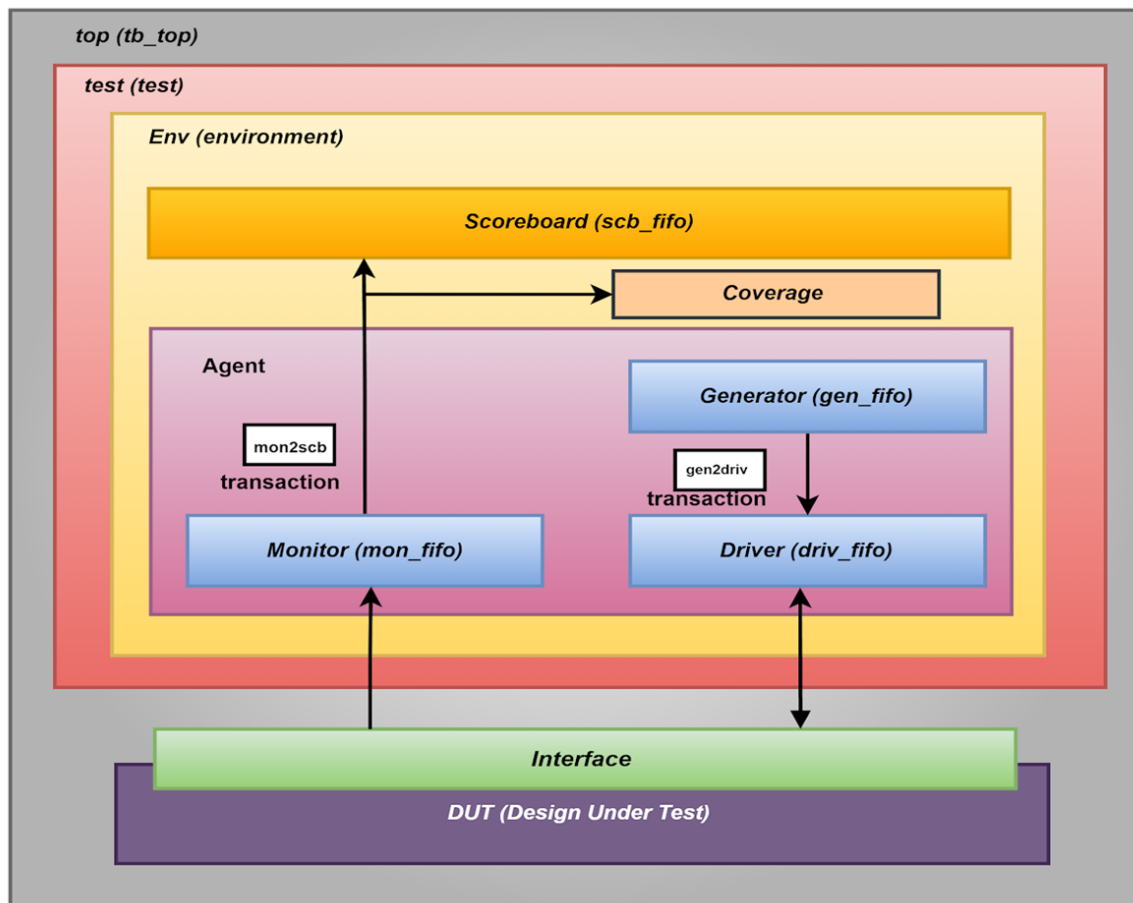


Figure 7: Block diagram of system Verilog environment

- In UVM, the entire uvm verification environment is constructed from the scratch with one sequence item, two sequences for read and write, a sequencer, agent, driver, monitor, scoreboard with coverage. The block diagram of uvm testbench is shown in fig. 8.

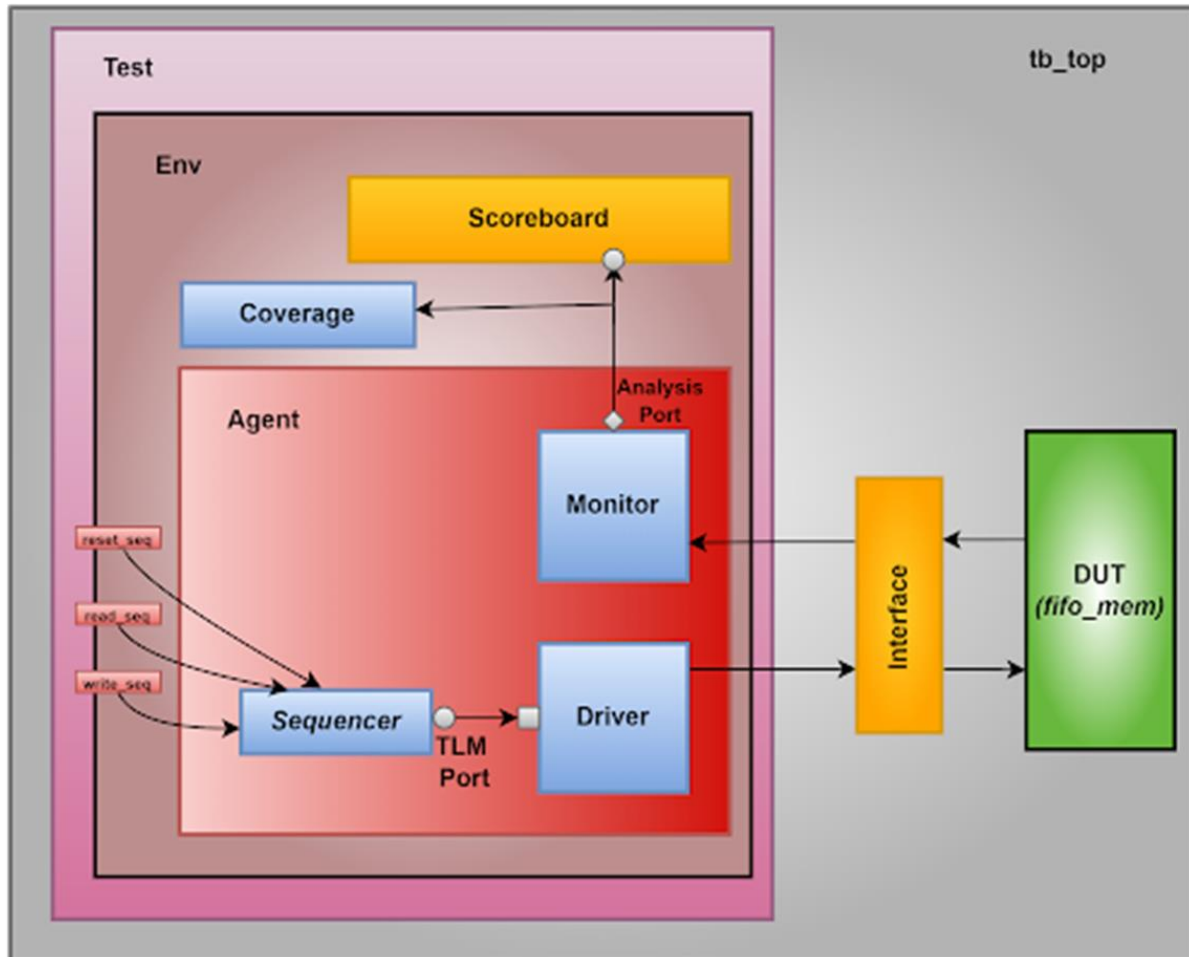


Figure 8: Block Diagram of UVM Verification Environment

6.1 Testbench Components

System Verilog testbench components are shown in table 1.

Table 1: Components of SystemVerilog environment

Name	Type	Description
transaction	class	Defines the pin level activity generated by agent (to drive to DUT through the driver) or the activity has to be observed by agent (Placeholder for the activity monitored by the monitor on DUT signals)
generator	class	Generates the stimulus (create and randomize the transaction class) and send it to Driver. r_en, w_en and data_in are generated here.
driver	class	Receives the stimulus (transaction) from a generator and drives the packet level data inside the transaction into pin level (to DUT. The data_in is sent to the DUT with proper w_en and r_en
monitor	class	Observes pin level activity on interface signals and converts into packet level which is sent to the components such as scoreboard
agent	class	An agent is a container class, which groups the class's (generator, driver, and monitor) specific to an interface or protocol
scoreboard	class	Receives data items from monitors and compares them with expected values. Expected values can be either golden reference values or generated from the reference model
environment	class	The environment is a container class for grouping higher level components like agent's and scoreboard
test	program	The test is responsible for configuring the testbench, initiate the testbench components construction process, initiate the stimulus driving.
testbench_top	class	This is the topmost file, which connects the DUT and Test Bench. It consists of DUT, Test and interface instances, the interface connects the DUT and Test Bench

The UVM components are shown in table 2

Table 2: Components of UVM Verification Environment

Name	Type	Description
UVM Sequence item	class	The sequence-item defines the pin level activity generated by agent (to drive to DUT through the driver) . Constraints for read enable, write enable are mentioned here.
UVM Sequence	class	Defines the sequence in which the data items need to be generated and sent/received to/from the driver. Read sequence and write sequence are used to generating the stimulus based on read enable and write enable.
UVM Sequencer	class	Responsible for routing the data_in(sequence_item) generated in sequence to the driver or vice verse.
UVM driver	class	Responsible for driving the packet level data inside sequence_item into pin level (to DUT).
UVM monitor	class	Observes pin level activity on interface signals and converts into packet level which is sent to components such as scoreboards.
UVM agent	class	An agent is a container class, which groups the class's (generator, driver, and monitor) specific to an interface or protocol
UVM scoreboard	class	Receives data items from monitors and compares them with expected values. Expected values can be either golden reference values or generated from the reference model
UVM environment	class	The environment is a container class for grouping higher level components like agent's and scoreboard
UVM test	program	The test is the topmost class responsible configuring the testbench, initiate the testbench components construction process by building the next level down in the hierarchy, initiate the stimulus by starting the sequence.
UVM testbench_top	class	This is the topmost file, which connects the DUT and Test Bench. It consists of DUT, Test and interface instances, the interface connects the DUT and Test Bench

6.2 Testbench Style

The constrained based randomized test bench style is used to test the asynchronous FIFO. In this style, constraints are applied to generate random stimuli for the design under test (DUT). The stimuli are generated within specified ranges and conditions to ensure coverage of various scenarios. Constrained random testbenches are effective for exploring corner cases and identifying potential bugs.

System Verilog

- The constraints for read enable and write enable are written in the transaction class in order to not assert the both signals at the same time.
- Weighted Constraint is written to equally generate the read and write enable.
- The rand variable the is declared for w_en, w_en and data_in in the transaction class.
- These signals are randomized using built-in randomize function in the generator.
- These signals are passed on to the driver using transaction class.
- The display statements are used to identify the flow of the transactions and data transfer in each component.

UVM

- The constraints for read enable and write enable are written in the sequence item in order to not assert the both signals at the same time.
- Weighted Constraint is written to equally generate the read and write enable.
- The rand variable the is declared for w_en, w_en and data_in in the sequence item.
- These signals are randomized using built-in randomize function in thesequence.
- These signals are passed on to the driver using sequence_item.
- The display statements are used to identify the flow of the transactions and data transfer in each component.

6.3 Testing Strategies

- **Functional Testing:** Testing the basic working functionality of the asynchronous fifo by checking the first in first out conditions between different clock domains.
- **Boundary Conditions:** Testing the full and empty conditions and checking the corner cases .

- **Random Data Transactions:** Generate random data transactions to test the FIFO's ability to handle arbitrary data sequences and Include data of different sizes to verify correct handling of various transaction lengths.
- **Asynchronous Operation Testing:** Verify correct operation under asynchronous conditions, where data is written and read at different clock domains or with different clock frequencies. Test for metastability scenarios, ensuring proper synchronization and recovery.
- **FIFO Depth Testing:** Test the FIFO with different depths to verify correct behavior and performance scalability and Evaluate FIFO behavior under various fill levels, including near-empty and near-full conditions.

6.4 Test case scenarios

- Write data into the FIFO and read it back to ensure correctness.
- Verify that the data is read out in the same order it was written.
- Test FIFO behavior when it is empty or full.
- Verify that writing to a full FIFO or reading from an empty FIFO produces the expected behavior.
- Test the FIFO under asynchronous conditions, where data is written and read at different clock domains or with different clock frequencies.
- Verify that the FIFO can handle maximum allowed widths and depths without errors.
- Apply stress conditions by continuously writing and reading data at high speeds to evaluate the FIFO's robustness and stability.
- Test FIFO reset functionality to ensure it clears internal state and returns the FIFO to a known state.
- Verify that resetting the FIFO does not cause data corruption or loss.
- Inject errors into the FIFO and verify that they are detected and handled correctly.
- Verify proper operation of handshaking signals (e.g., write enable, read enable) to ensure correct synchronization between the FIFO and its interfacing components.

6.5 Self checking with scoreboard

The self-checking of the fifo is done by creating the reference queue. First the data_in is pushed front into the reference queue and the data from the queue is popped back to the check_fifo. The data from the check_fifo is compared with the data_out and the burst details are displayed with the score board transfer number stating whether the tests are passes or failed.

7. Simulation Results

7.1 System Verilog Simulation Results

Data passing through different components of the testbench

```
# .....[Driver] write reset ended.....
# .....[Driver] read reset started.....
# .....[Driver] read reset ended.....
# [Generator][Generator trasaction no=0]:---data_in = 48, w_en = 1, r_en =0
# [Generator][Generator trasaction no=1]:---data_in = 10, w_en = 1, r_en =0
# [Generator][Generator trasaction no=2]:---data_in = 53, w_en = 1, r_en =0
# [Generator][Generator trasaction no=3]:---data_in = 77, w_en = 1, r_en =0
# [Generator][Generator trasaction no=4]:---data_in =167, w_en = 0, r_en =1
# [Generator][Generator trasaction no=5]:---data_in =248, w_en = 0, r_en =1
# [Generator][Generator trasaction no=6]:---data_in = 68, w_en = 0, r_en =1
# [Generator][Generator trasaction no=7]:---data_in =215, w_en = 0, r_en =1
# -----INSIDE DRIVER: DRIVE TASK-----
# -----INSIDE MONITOR MAIN TASK-----
# -----INSIDE SCOREBOARD:MAIN TASK-----
# 102[Driver][Driver Transfer : 0]----- w_en = 1,r_en=0, waddr = 1, data_in = 48
# 102[MONITOR][Monitor Write Transfer : 0]----w_en = 1,r_en = 0, data_in = 48, empty = 0,full = 0,waddr = 1
# 102[SCB-PASS][Scb-transfer =0] Scoreboard data written
# -----
# 106[Driver][Driver Transfer : 1]----- w_en = 1,r_en=0, waddr = 2, data_in = 10
# 106[MONITOR][Monitor Write Transfer : 1]----w_en = 1,r_en = 0, data_in = 10, empty = 0,full = 0,waddr = 2
# 106[SCB-PASS][Scb-transfer =1] Scoreboard data written
# -----
# 110[Driver][Driver Transfer : 2]----- w_en = 1,r_en=0, waddr = 3, data_in = 53
# 110[MONITOR][Monitor Write Transfer : 2]----w_en = 1,r_en = 0, data_in = 53, empty = 0,full = 0,waddr = 3
# 110[SCB-PASS][Scb-transfer =2] Scoreboard data written
# -----
# 114[Driver][Driver Transfer : 3]----- w_en = 1,r_en=0, waddr = 4, data_in = 77
# 114[MONITOR][Monitor Write Transfer : 3]----w_en = 1,r_en = 0, data_in = 77, empty = 0,full = 0,waddr = 4
# 114[SCB-PASS][Scb-transfer =3] Scoreboard data written
# -----
```

FIFO Full Condition

```
-----
1118[Driver][Driver Transfer : 254]----- w_en = 1,r_en=0, waddr = 255, data_in = 205
1118[MONITOR][Monitor Write Transfer : 254]----w_en = 1,r_en = 0, data_in = 205, empty = 0,full = 0,waddr = 255
1118[SCB-PASS][Scb-transfer =254] Scoreboard data written
-----
1122[Driver][Driver Transfer : 255]----- w_en = 1,r_en=0, waddr = 256, data_in = 20
1122[MONITOR][Monitor Write Transfer : 255]----w_en = 1,r_en = 0, data_in = 20, empty = 0,full = 1,waddr = 256
-----
```

FIFO Empty Condition

```
-----
3665[Driver][Driver Transfer : 510]-----r_en = 1,w_en = 0, raddr = 255
3665[MONITOR][Monitor Read Transfer : 509]----w_en = 0,r_en = 1,empty = 0,full = 0,data_out = 20,raddr = 255
3665[SCB-PASS][Scb-transfer =509] Test Passed successfully
-----
3675[Driver][Driver Transfer : 511]-----r_en = 1,w_en = 0, raddr = 256
3675[MONITOR][Monitor Read Transfer : 510]----w_en = 0,r_en = 1,empty = 1,full = 0,data_out = 48,raddr = 256
-----
** Note: $finish      : environment.sv(66)
    Time: 3675 ns   Iteration: 2   Instance: /tb_top/t1
1
```

7.2 UVM simulation results

UVM Report Summary

```

# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 3631
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [AGENT CLASS] 4
# [COVERAGE] 2
# [COVERAGE CLASS] 1
# [Comparing read] 514
# [DRIVER] 1
# [DRIVER CLASS] 3
# [DRIVER_READ] 256
# [DRIVER_WRITE] 256
# [ENVIRONMENT CLASS] 4
# [FIFO_READ_SEQUENCE] 257
# [FIFO_SEQUENCE] 259
# [MONITOR] 4
# [MONITOR READ] 257
# [MONITOR WRITE] 258
# [Questa UVM] 2
# [RNTST] 1
# [SCOREBOARD] 4
# [SEQ] 1027
# [SEQUENCER] 3
# [Scoreboard write data_in] 256
# [Scoreboard getting read data_in] 256
# [TEST CLASS] 4
# [TEST_DONE] 1
# [UVMTOP] 1
# ** Note: $finish : C:/questasim64_2023.3/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 3605 ns Iteration: 54 Instance: /uvmtb_top
# 1

```

FIFO Full condition

```

# -----
# UVM_INFO monitor.sv(52) @ 1042: uvm_test_top.e0.agnt.mon [MONITOR WRITE] Burtst Dtails:time=1042,w_en=0,r_en=1,data_in=166,full=1,empty=0, waddr=256,
# UVM_INFO monitor.sv(66) @ 1045: uvm_test_top.e0.agnt.mon [MONITOR READ] Burtst Dtails:time=1045,w_en=0,r_en=1,data_out=107,full=1,empty=0, raddr= 0
# UVM_INFO driver.sv(77) @ 1045: uvm_test_top.e0.agnt.d0 [DRIVER_READ] Burtst Dtails:time=1045,w_en=0,r_en=1,data_out=107,full=1,empty=0,raddr= 0
# UVM_INFO scoreboard.sv(68) @ 1045: uvm_test_top.e0.scb [Scoreboard getting read data_in] Burtst Dtails:r_en=1, data_in=166, full=0
# UVM_INFO scoreboard.sv(76) @ 1045: uvm_test_top.e0.scb [Comparing read] transaction passed actual=107, expected_data=107
# UVM_INFO sequence_fifo_wr.sv(93) @ 1045: reporter@@sequence_fifo_rd [SEQ] done Generate new item: 0
# UVM_INFO sequence_fifo_wr.sv(84) @ 1045: reporter@@sequence_fifo_rd [FIFO_READ_SEQUENCE] INSIDE THE TASK BODY!
# UVM_INFO sequence_fifo_wr.sv(88) @ 1045: reporter@@sequence_fifo_rd [SEQ] Generate new item:w_en =0, r_en =1,data_in =38
# -----

```

FIFO Empty Condition

```

# -----
# UVM_INFO driver.sv(77) @ 3595: uvm_test_top.e0.agnt.d0 [DRIVER_READ] Burtst Dtails:time=3595,w_en=0,r_en=1,data_out=109,full=0,empty=0,raddr=255
# UVM_INFO monitor.sv(66) @ 3595: uvm_test_top.e0.agnt.mon [MONITOR READ] Burtst Dtails:time=3595,w_en=0,r_en=1,data_out=109,full=0,empty=0, raddr=255
# UVM_INFO sequence_fifo_wr.sv(93) @ 3595: reporter@@sequence_fifo_rd [SEQ] done Generate new item: 255
# UVM_INFO sequence_fifo_wr.sv(95) @ 3595: reporter@@sequence_fifo_rd [SEQ] Done Generation of items: 256
# UVM_INFO scoreboard.sv(68) @ 3595: uvm_test_top.e0.scb [Scoreboard getting read data_in] Burtst Dtails:r_en=1, data_in=229, full=0
# UVM_INFO scoreboard.sv(76) @ 3595: uvm_test_top.e0.scb [Comparing read] transaction passed actual=109, expected_data=109
# UVM_INFO monitor.sv(66) @ 3605: uvm_test_top.e0.agnt.mon [MONITOR READ] Burtst Dtails:time=3605,w_en=0,r_en=1,data_out=107,full=0,empty=1, raddr=256
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 3605: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO scoreboard.sv(76) @ 3605: uvm_test_top.e0.scb [Comparing read] transaction passed actual= 0, expected_data= 0

```


SUMMARY

The High-Level Design Specifications (HLDS) document outlines the detailed description of an Asynchronous FIFO (First-In-First-Out) for documentation and maintainability purposes. It serves as a reference guide for module interactions, aiding in conflict identification before coding. The document includes design constraints, assumptions, dependencies, and various modules like Write Pointer, Read Pointer, FIFO Memory, and synchronizers. The asynchronous FIFO employs a pointer-based method to detect full and empty conditions, facilitating low access latency. Additionally, it utilizes FIFO memory to manage data in a sequential manner and synchronizers to mitigate metastability risks between clock domains. The design consists of five modules interconnected as per functional requirements, with connections managed by the top-level wrapper module. Testing strategies include functional testing, boundary conditions testing, random data transactions, and asynchronous operation testing. Test case scenarios encompass various scenarios like data integrity, asynchronous operation, stress testing, and error handling. Self-checking with a scoreboard involves comparing data outputs with a reference queue to ensure correct operation. Overall, the document provides a comprehensive overview of the Asynchronous FIFO design and its testing methodologies.

APPENDIX

A. Glossary

1. **Asynchronous FIFO:** A type of First-In-First-Out memory buffer that allows data to be written and read asynchronously, meaning it can handle data transfer between different clock domains without requiring synchronization between them.
2. **FIFO (First-In-First-Out):** A data structure that operates on the principle of first data in being the first data out. In the context of asynchronous FIFOs, it refers to a memory buffer that stores data in the order it was received and retrieves it in the same order.
3. **Clock Domain:** A distinct region of a digital circuit that operates on its own clock signal. Asynchronous FIFOs often involve data transfer between different clock domains, necessitating special considerations for synchronization.
4. **Metastability:** A transient state that occurs in digital circuits when signals change close to the edge of a clock cycle, potentially leading to unpredictable behavior. Synchronizers are used to mitigate the effects of metastability when signals traverse between different clock domains.
5. **Pointer:** A variable or register used to track the position or status of data within a FIFO. In asynchronous FIFOs, pointers are utilized to indicate where data is being written and read, helping to determine full and empty conditions.
6. **Synchronizer:** A circuit or mechanism used to synchronize signals between different clock domains, reducing the risk of metastability. In the context of asynchronous FIFOs, synchronizers ensure reliable data transfer between asynchronous components.
7. **MTBF (Mean Time Between Failures):** A reliability metric used to estimate the average time elapsed between failures in a system or component. MTBF is calculated by dividing the total operating time by the number of failures observed within that time period. It is commonly used to assess the reliability and durability of electronic systems.
8. **Clock Domain Crossing (CDC):** The process of transferring signals between different clock domains in a digital circuit. CDC occurs when signals originating from one clock domain are received and processed in another clock domain with a different clock frequency. Managing

clock domain crossings is crucial to ensure proper synchronization and prevent issues such as metastability and data corruption. Techniques like synchronizers are employed to safely transfer signals across clock domains.

9. **UVM** (Universal Verification Methodology): A standardized methodology for verifying and validating digital designs using SystemVerilog. UVM provides a framework for creating modular, reusable, and scalable test environments, allowing efficient verification of complex designs.
10. **Reset**: A signal used to initialize or reset the state of a circuit. Asynchronous FIFOs often utilize reset signals to clear internal states and return the FIFO to a known state.
11. **RTL** (Register-Transfer Level): A level of abstraction in digital circuit design that represents the behavior of a circuit in terms of registers and the transfer of data between them. RTL modules describe the functionality of individual components within the asynchronous FIFO design.
12. **Simulation**: The process of modeling and testing a digital circuit's behavior using simulation tools. Simulation is crucial for verifying the functionality and performance of asynchronous FIFO designs under various conditions.

This glossary provides definitions for key terms and concepts mentioned in the description of the asynchronous FIFO design, aiding readers in understanding the technical terminology and specialized vocabulary used in the document.

B. Asynchronous fifo depth calculation

FIFO should be able operate under following conditions:

- ☐ Producer clk1 freq = 250 Mhz
- ☐ Consumer clk2 freq = 100Mhz
- ☐ With duty-cycle of 50%
- ☐ Max write Burst size = 150
- ☐ Num. of ideal cycle between successive writes = 0
- ☐ Num. of ideal cycles between successive reads = 2

Solution:

- The no. of idle cycles between two successive writes is 0 clock cycle.
- It means that, after writing one data, module A is not waiting for any clock cycle, to initiate the next write. So, it can be understood that for every clock cycles, one data is written.
- The no. of idle cycles between two successive reads is 2 clock cycles. It means that, after reading one data, module B is waiting for 2 clock cycles, to initiate the next read. So, it can be understood that for every 3 clock cycles, one data is read.
- Time required to write one data item = $1 * 1/250\text{MHz} = 4\text{nSec}$
- Time required to write all the data in the burst = $150 * 4 \text{ nSec.} = 600 \text{ nSec.}$
- Time required to read one data item = $3 * 1/100\text{MHz} = 30\text{ns}$
- So, for every 30 nSec, the module B is going to read one data item in the burst.
- So, in a period of 600 nSec, 150 no. of data items can be written.
- The no. of data items can be read in a period of 600 nSec = $600\text{nsec}/30\text{nsec} = 20$
- The remaining no. of bytes to be stored in the FIFO = $150 - 20 = 130$. So, the FIFO which has to be in this scenario must be capable of storing 130 data items.

So, the minimum depth of the FIFO should be 130.